Knowledge Center

XXE Complete Guide: Impact, Examples, and Prevention

What Is an XXE (XML External Entity) Vulnerability?

XML External Entity (XXE) is an application-layer cybersecurity attack that exploits an XXE vulnerability to parse XML input. XXE attacks are possible when a poorly configured parser processes XML input with a pathway to an external entity. This can damage organizations in various ways, including denial of service (DoS), sensitive data exposure, server-side request forgery (SSRF), and port scanning from the parser's locations.

XML documents are defined using the XML 1.0 standard, which includes the concept of an "entity" that stores data. Several kinds of entities can access data locally or remotely through a system identifier. An external entity, or external general parameter-parsed entity, can request and receive data, including confidential data.

The XRL processing system assumes that the declared system identifier is an accessible URL. When processing a named entity, the processor replaces each entity instance with the dereferenced contents from the identifier. If these contents include flawed or manipulated data, the XML processor will dereference (access) this data, potentially disclosing sensitive information to the external entity. This technique allows applications to access otherwise protected data.

Other related attack types use similar vectors to include external resources in the internal processing of an application. These attacks may use external document type definitions (DTDs), stylesheets, or schemas.

What Is the Impact of XXE Injections?

XXE attacks can have an impact both on the vulnerable application, and on other systems it is connected to.

On the targeted application, attackers may be able to retrieve sensitive data such as passwords, or perform directory traversal to gain access to sensitive paths on the local server. XXE can also be used to perform a type of denial of service (DoS) attack by accessing a large number of resources or opening too many threads on the local server.

On other connected systems, attackers might leverage their access to the targeted application to gain access to other directories on the network, perform port scanning, or carry out server side request forgery (SSRF) attacks.

In extreme cases, an XML processor library might be vulnerable to client-side memory corruption issues, which may allow remote code execution under the application's privileges.

5 Examples of XXE Attack Payloads

Resource Exhaustion Attacks

The most basic XML-based attack, although not strictly an external XML entity attack, is the so-called "billion laughs" attack. This attack is mitigated in most modern XML parsers, but can help illustrate how XML attacks work.

Take the following DOCTYPE definition that defines a new XML entity:

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE laugh [
   <!ELEMENT laugh ANY>
   <!ENTITY LOL "LOL">
   <!ENTITY LOL1 "&LOL1;&LOL1;&LOL1;&LOL1;&LOL1;&LOL1;&LOL1;">
   <!ENTITY LOL2 "&LOL2;&LOL2;&LOL2;&LOL2;&LOL2;&LOL2;&LOL2;">
   <!ENTITY LOL3 "&LOL3;&LOL3;&LOL3;&LOL3;&LOL3;&LOL3;&LOL3;">
]>
<laugh>&LOL3;</laugh>
```

The XML parser parses this code and expands each of the entities, generating a large number of "LOLs".

The above example generates several hundred LOL strings, but in a full-scale example, the code could generate billions of lines of output, exhausting memory on the server. An alternative way to achieve the same effect is to reference a very long or infinite string, such as the /dev/urandom string on Linux operating systems.

Data Extraction Attacks

XML attacks get more interesting when external entities are involved. An external entity (defined on a server controlled by the attacker) can reference URIs on the local server to retrieve sensitive content from the file system. Most servers use the same directories for sensitive system files, making this an easy endeavor for attackers.

For example, the following code will return the content of the login.defs file, which defines login settings, on a vulnerable Linux system:

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE malicious [
   <!ELEMENT malicious ANY>
   <!ENTITY external SYSTEM "file:////etc/login.defs">
]>
<malicious>&external;</malicious>
```

Another important element of XXE attacks is that they can be used to scan ports or retrieve data from other hosts connected to the target system. For example, if the target system can connect to a file server on IP address 10.0.0.5, the attacker can retrieve sensitive data from the server like this:

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE malicious [
   <!ELEMENT malicious ANY>
   <!ENTITY external SYSTEM "http://10.0.0.5/sensitive.txt">
]>
<malicious>&external;</malicious>
```

Related content: Read our guide to data breaches

SSRF Attacks

Attackers can use XXE attacks for more than just retrieving sensitive data. Another possible impact is that XXE can be used to perform server-side request forgery (SSRF). An SSRF attack involves attackers exploiting a server-side application to make HTTP requests to any URL that the server can reach.

In order to perform an SSRF attack via an XXE vulnerability, the attacker needs to define an external XML entity with the target URL they want to reach from the server, and use this entity in a data value. If the attacker manages to place this data value within an application response, they will be able to see the content of the URL within the app response, allowing two-way interaction with the backend system. If an application response is not available, the attackers can still perform a blind SSRF attack.

Here is an example of an external entity that causes a server to make a backend HTTP request to an internal system within the organization's network:

<!DOCTYPE malicious [<!ENTITY external SYSTEM "http://sensitive-system.company.com/">]>

Related content: Read our guide to SSRF

File Retrieval

Attackers exploit XXE to retrieve files that contain an external entity definition of the file's contents. The application sends the files in its response. To perform this type of XXE injection attack and retrieve arbitrary files from a server's file system, the attacker must modify the XML by:

- Introducing or editing a DOCTYPE element defining an entity with a path to the target file.
- Editing the data values in the submitted XML, returned by the application, and using the external entity it defines.

Blind XXE

Attackers exploit blind XXE vulnerabilities to retrieve or exfiltrate data. For example, attackers can steal out-of-band data, inducing the application server to send sensitive data to an external system under their control.

An attacker might also exploit blind XXE to receive error messages containing sensitive data. The attacker triggers parsing error messages that expose the data.

How to Prevent XML External Entity Injections

Here are two common ways to prevent XXE attacks in your organization.

Managed WAF with Custom-Defined Rules

A web application firewall (WAF) defends the application layer (network Layer 7) of an organization's network perimeter. It intercepts incoming and outgoing traffic of websites and web applications. A WAF monitors and filters incoming and outgoing data packets and HTTP requests, blocking packets or requests that match known attack patterns or specific rules defined by the WAF administrator.

Most WAF solutions have built-in rules that can block obvious XXE inputs. Advanced WAF products can additionally detect non-obvious XXE attacks, using behavioral analysis to

understand which XML entities seem suspicious or exhibit unusual behavior. Combined with allowlists and denylists defined by the WAF administrator, this can provide a good solution for XXE vulnerabilities, even without remediating the underlying vulnerable components.

Application Server Instrumentation

Application server instrumentation (ASI) is a technology that monitors the flow of execution at runtime by injecting checkpoints into specific parts of application code. Adding a security sensor to your server gives you real-time visibility into the application schema and the data flow for every request. Instrumentation is a core component of dynamic testing solutions such as runtime application self protection (RASP) and interactive application security testing (IAST).

Instrumentation can be very useful in preventing XXE attacks. ASI can monitor key classes involved in XML processing and validate any activity related to remote DTDs. The XML parser could be part of your application's third-party code, and due to the sheer number of components in modern applications, you cannot rely on manual configuration. Instrumentation eliminates the need for manual validation, automatically detecting XXE vulnerabilities.

In addition to detecting attacks, instrumentation can actively prevent attacks. For example, it can prevent execution of external code via XML entities, and rate limit XML-related requests, significantly reducing the risk of XXE-related DoS attacks.

Conclusion

In this article, we explained how XXE attacks work, and covered the following types of XXE attack payloads:

- **Resource exhaustion attacks** - attacks that recursively repeat content using XML entities, leading to denial of service on the server.
- **Data extraction attacks** - attacks that use external entities to access URIs on the server or connected systems.
- **SSRF attacks** - attacks in which an external entity is used to make HTTP requests from a server-side application to another URL the server can reach.
- **File retrieval** - attacks in which an external entity is used to directly retrieve sensitive files from the server.
- **Blind XXE** - attacks that induce the application server to send sensitive data to an external system under the attacker’s control, or display sensitive data within error messages.

Finally, we reviewed two approaches to XXE prevention - configuring custom WAF rules to block XXE communications, and performing application server instrumentation.